

# 03 — Transactions in Applications

## Table of Contents

- 1. [Learning Objectives](#)
- 2. [Transaction Fundamentals Recap](#)
- 3. [Python — psycopg2](#)
- 4. [Java — Spring @Transactional](#)
- 5. [Node.js — pg.Transaction Blocks](#)
- 6. [Nested Transactions and SAVEPOINTS](#)
- 7. [Retry Logic for Serialization Failures](#)
- 8. [Transaction Timeout Patterns](#)
- 9. [Common Mistakes](#)
- 10. [Best Practices](#)
- 11. [Performance Considerations](#)
- 12. [Interview Questions & Answers](#)
- 13. [Exercises with Solutions](#)
- 14. [Cross-References](#)

## Learning Objectives

By the end of this section you will be able to:

- Manage PostgreSQL transactions in Python (psycopg2 autocommit and context manager)
- Use Spring @Transactional with correct propagation levels
- Write robust transaction blocks in Node.js with error handling
- Implement nested transactions using SAVEPOINT
- Build retry logic for serialization failures (SQLSTATE 40001)
- Set and enforce transaction timeouts in application code

## Transaction Fundamentals Recap

### ACID Properties

| Property    | Meaning                                                        | PostgreSQL Mechanism                     |
|-------------|----------------------------------------------------------------|------------------------------------------|
| Atomicity   | All-or-nothing                                                 | WAL + rollback                           |
| Consistency | Constraints always satisfied                                   | Constraints + triggers checked at COMMIT |
| Isolation   | Concurrent transactions don't see each other's partial changes | MVCC                                     |
| Durability  | Committed data survives crashes                                | WAL flush to disk                        |

### Isolation Levels in PostgreSQL

```
-- PostgreSQL supports 4 standard levels (but RU = RC in practice)
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED; -- default
```

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- READ UNCOMMITTED maps to READ COMMITTED in PostgreSQL
```

| Level              | Dirty Read | Non-Repeatable Read | Phantom Read | Serialization Anomaly |
|--------------------|------------|---------------------|--------------|-----------------------|
| Read Committed     | No         | Possible            | Possible     | Possible              |
| Repeatable Read    | No         | No                  | No (MVCC)    | Possible              |
| Serializable (SSI) | No         | No                  | No           | No                    |

## Python — psycopg2

### Autocommit Mode

```
import psycopg2

conn = psycopg2.connect("dbname=myapp user=app host=localhost")

# Default: autocommit=False – psycopg2 wraps everything in a transaction
conn.autocommit = False # (default)

# Every statement is implicitly in a transaction
cursor = conn.cursor()
cursor.execute("INSERT INTO orders (customer_id, total) VALUES (%s, %s)", (123,
99.99))
conn.commit() # explicit commit required
# or
conn.rollback() # explicit rollback on error

# Enable autocommit for DDL or single-statement operations
conn.autocommit = True
cursor.execute("CREATE INDEX CONCURRENTLY idx_orders_created ON orders(created_at)")
# ^ CREATE INDEX CONCURRENTLY cannot run inside a transaction
```

### Context Manager (Recommended Pattern)

```
import psycopg2
import psycopg2.extras
from contextlib import contextmanager

# Connection context manager
with psycopg2.connect("dbname=myapp user=app") as conn:
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute("INSERT INTO orders (customer_id, total) VALUES (%s, %s)", (123,
```

```

99.99))
    cur.execute("UPDATE inventory SET quantity = quantity - 1 WHERE product_id =
%s", (456,))
    # conn.__exit__ calls commit() on success, rollback() on exception
# Note: conn is NOT closed by the with block – only committed/rolled back

# Explicit close
conn.close()

```

## Transaction with Error Handling

```

def create_order(conn, customer_id: int, items: list[dict]) -> int:
    """Create an order with multiple line items atomically."""
    try:
        with conn.cursor() as cur:
            # Insert order header
            cur.execute("""
                INSERT INTO orders (customer_id, status, created_at)
                VALUES (%s, 'pending', NOW())
                RETURNING id
            """, (customer_id,))
            order_id = cur.fetchone()[0]

            # Insert line items
            psycopg2.extras.execute_values(cur, """
                INSERT INTO order_items (order_id, product_id, quantity, unit_price)
                VALUES %s
            """, [(order_id, item['product_id'], item['qty'], item['price'])
                  for item in items])

            # Deduct inventory
            for item in items:
                cur.execute("""
                    UPDATE inventory
                    SET quantity = quantity - %s
                    WHERE product_id = %s AND quantity >= %s
                """, (item['qty'], item['product_id'], item['qty']))
                if cur.rowcount == 0:
                    raise ValueError(f"Insufficient stock for product
{item['product_id']}")

            conn.commit()
            return order_id

    except Exception as e:
        conn.rollback()
        raise

```

## psycopg3 (psycopg) Improvements

```

import psycopg

# psycopg3: async by default, cleaner API
async def create_order_async(conn_info: str, customer_id: int, items: list) -> int:
    async with await psycopg.AsyncConnection.connect(conn_info) as conn:
        async with conn.transaction(): # auto commit/rollback
            async with conn.cursor() as cur:
                await cur.execute(
                    "INSERT INTO orders (customer_id) VALUES (%s) RETURNING id",
                    (customer_id,)
                )
                order_id = (await cur.fetchone())[0]
                # ... rest of inserts
            return order_id

# psycopg3: pipeline mode (send multiple queries without waiting)
async with await psycopg.AsyncConnection.connect(conn_info) as conn:
    async with conn.pipeline():
        await conn.execute("UPDATE accounts SET balance = balance - %s WHERE id = %s", (100, 1))
        await conn.execute("UPDATE accounts SET balance = balance + %s WHERE id = %s", (100, 2))
    # Both sent in a single round-trip, committed together

```

## Java — Spring @Transactional

### Basic Usage

```

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private InventoryRepository inventoryRepository;

    @Transactional // default: REQUIRED, READ_COMMITTED, rollback on
    RuntimeException
    public Order createOrder(Long customerId, List<OrderItem> items) {
        Order order = new Order();
        order.setCustomerId(customerId);
        order.setStatus("pending");
        order = orderRepository.save(order);

        for (OrderItem item : items) {
            item.setOrder(order);
            orderRepository.saveItem(item);
        }
    }
}

```

```

        int updated = inventoryRepository.deductStock(item.getProductId(),
item.getQuantity());
        if (updated == 0) {
            throw new InsufficientStockException(item.getProductId()); //
triggers rollback
        }
    }
    return order;
}
}

```

## Propagation Levels

```

// REQUIRED (default): join existing transaction or create new one
@Transactional(propagation = Propagation.REQUIRED)
public void methodA() { methodB(); } // methodB runs in same transaction

// REQUIRES_NEW: always start a new transaction, suspend current one
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void auditLog(String action) {
    // runs in its OWN transaction – commits even if caller rolls back
    auditRepository.save(new AuditEntry(action));
}

// NESTED: use SAVEPOINT – rollback to savepoint without rolling back outer
transaction
@Transactional(propagation = Propagation.NESTED)
public void tryOptionalStep() {
    // if this throws, only rolls back to the savepoint
    // outer transaction continues
}

// MANDATORY: must be called within an existing transaction
@Transactional(propagation = Propagation.MANDATORY)
public void mustBeInTransaction() { /* ... */ }

// NEVER: must NOT be called within a transaction
@Transactional(propagation = Propagation.NEVER)
public void noTransactionAllowed() { /* ... */ }

// NOT_SUPPORTED: suspend current transaction, run non-transactionally
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void longRunningReport() { /* ... */ }

// SUPPORTS: run in transaction if one exists, otherwise non-transactional
@Transactional(propagation = Propagation.SUPPORTS)
public List<Order> listOrders() { /* ... */ }

```

## Isolation Levels in Spring

```
@Transactional(isolation = Isolation.READ_COMMITTED)    // default in PostgreSQL
@Transactional(isolation = Isolation.REPEATABLE_READ)
@Transactional(isolation = Isolation.SERIALIZABLE)
```

## Rollback Rules

```
// Default: rolls back on RuntimeException (unchecked), NOT on checked exceptions
@Transactional
public void process() throws CheckedException {
    // CheckedException does NOT trigger rollback by default!
}

// Explicit rollback on checked exception:
@Transactional(rollbackFor = {CheckedException.class, AnotherException.class})
public void process() throws CheckedException { /* ... */ }

// Do NOT rollback on certain exceptions:
@Transactional(noRollbackFor = CacheException.class)
public void process() { /* ... */ }
```

## Common @Transactional Pitfalls

```
@Service
public class OrderService {

    // PITFALL 1: Self-invocation – @Transactional doesn't work on internal calls
    public void publicMethod() {
        this.transactionalMethod(); // Spring proxy NOT involved – no transaction!
    }

    @Transactional
    public void transactionalMethod() { /* ... */ }

    // FIX: inject self or extract to another bean
    @Autowired
    private OrderService self;

    public void publicMethod() {
        self.transactionalMethod(); // proxy IS involved
    }

    // PITFALL 2: @Transactional on private methods – ignored by Spring AOP
    @Transactional
    private void privateMethod() { /* NO TRANSACTION */ }

    // PITFALL 3: Transaction spans too much – holding connection while calling
    // external APIs
    @Transactional
    public void badPattern() {
```

```

        Order order = orderRepository.save(new Order());
        paymentService.callExternalAPI(order.getId()); // 500ms network call holding
DB connection
        orderRepository.updateStatus(order.getId(), "paid");
    }

    // FIX: minimize transaction scope
    public void goodPattern() {
        Order order = orderRepository.save(new Order()); // small transaction
        String paymentId = paymentService.callExternalAPI(order.getId()); // outside
TX
        updateOrderStatus(order.getId(), paymentId); // small transaction
    }

    @Transactional
    public void updateOrderStatus(Long orderId, String paymentId) { /* ... */ }
}

```

## Reading Transactional Data (Read-Only Optimization)

```

@Transactional(readOnly = true) // tells Spring and driver: no writes expected
public List<Order> getOrdersByCustomer(Long customerId) {
    // PostgreSQL: read-only transaction can use standby replica
    // Hibernate: skips dirty checking, flush is disabled – faster
    return orderRepository.findByCustomerId(customerId);
}

```

## Node.js — pg Transaction Blocks

### Basic Transaction with node-postgres

```

const { Pool } = require('pg');
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

async function createOrder(customerId, items) {
    const client = await pool.connect();
    try {
        await client.query('BEGIN');

        const orderResult = await client.query(
            'INSERT INTO orders (customer_id, status) VALUES ($1, $2) RETURNING id',
            [customerId, 'pending']
        );
        const orderId = orderResult.rows[0].id;

        for (const item of items) {
            await client.query(
                'INSERT INTO order_items (order_id, product_id, quantity) VALUES
($1, $2, $3)',

```

```

        [orderId, item.productId, item.quantity]
    );

    const inv = await client.query(
        'UPDATE inventory SET qty = qty - $1 WHERE product_id = $2 AND qty
    >= $1 RETURNING qty',
        [item.quantity, item.productId]
    );
    if (inv.rowCount === 0) {
        throw new Error(`Insufficient stock: product ${item.productId}`);
    }
}

    await client.query('COMMIT');
    return orderId;
} catch (err) {
    await client.query('ROLLBACK');
    throw err;
} finally {
    client.release(); // ALWAYS release back to pool
}
}

```

## Transaction Helper Utility

```

// utils/transaction.js
async function withTransaction(pool, callback) {
    const client = await pool.connect();
    try {
        await client.query('BEGIN');
        const result = await callback(client);
        await client.query('COMMIT');
        return result;
    } catch (err) {
        await client.query('ROLLBACK');
        throw err;
    } finally {
        client.release();
    }
}

// Usage
const orderId = await withTransaction(pool, async (client) => {
    const { rows } = await client.query(
        'INSERT INTO orders (customer_id) VALUES ($1) RETURNING id',
        [customerId]
    );
    return rows[0].id;
});

```



## Isolation Level in Node.js

```
// Set isolation level immediately after BEGIN
await client.query('BEGIN ISOLATION LEVEL REPEATABLE READ');

// Or as a single statement
await client.query('BEGIN; SET TRANSACTION ISOLATION LEVEL SERIALIZABLE');
// Note: these are two statements – use semicolons or separate queries
await client.query('BEGIN');
await client.query('SET TRANSACTION ISOLATION LEVEL SERIALIZABLE');
```

## Nested Transactions and SAVEPOINTS

### What Is a SAVEPOINT?

A `SAVEPOINT` marks a point within a transaction to which you can roll back without aborting the entire transaction. PostgreSQL fully supports them.

```
BEGIN;
    INSERT INTO orders (customer_id) VALUES (1);           -- Step A

    SAVEPOINT sp1;
    INSERT INTO order_items (order_id, product_id) VALUES (1, 999); -- Step B (may fail)

    -- If Step B fails:
    ROLLBACK TO SAVEPOINT sp1; -- undoes Step B, Step A still in place
    -- Now try alternative:
    INSERT INTO order_notes (order_id, note) VALUES (1, 'item unavailable');

    RELEASE SAVEPOINT sp1; -- optional: remove savepoint (frees small memory)
COMMIT;
```

### Python SAVEPOINTS

```
def create_order_with_fallback(conn, customer_id, items):
    with conn.cursor() as cur:
        cur.execute("INSERT INTO orders (customer_id) VALUES (%s) RETURNING id",
            (customer_id,))
        order_id = cur.fetchone()[0]

        for item in items:
            cur.execute("SAVEPOINT item_sp")
            try:
                cur.execute("""
                    INSERT INTO order_items (order_id, product_id, quantity)
                    VALUES (%s, %s, %s)
                    """, (order_id, item['product_id'], item['qty']))
```

```

        cur.execute("""
            UPDATE inventory SET quantity = quantity - %s
            WHERE product_id = %s AND quantity >= %s
            """, (item['qty'], item['product_id'], item['qty']))

        if cur.rowcount == 0:
            raise ValueError("Out of stock")

        cur.execute("RELEASE SAVEPOINT item_sp")

    except ValueError:
        cur.execute("ROLLBACK TO SAVEPOINT item_sp")
        # Log the failed item, continue with the rest
        print(f"Skipping out-of-stock item {item['product_id']}")

    conn.commit()
    return order_id

```

## Spring Nested Transactions (SAVEPOINT)

```

@Service
public class OrderProcessor {

    @Transactional
    public ProcessResult processOrder(Order order) {
        orderRepository.save(order);

        for (OrderItem item : order.getItems()) {
            try {
                inventoryService.reserveItem(item); // REQUIRES_NEW or NESTED
            } catch (OutOfStockException e) {
                // With NESTED: only this savepoint rolls back
                item.setStatus("backordered");
                backorderService.createBackorder(item);
            }
        }

        return new ProcessResult(order);
    }
}

@Service
public class InventoryService {

    @Transactional(propagation = Propagation.NESTED)
    public void reserveItem(OrderItem item) {
        // Runs within a SAVEPOINT in the parent transaction
        inventory.deduct(item.getProductId(), item.getQuantity());
        // If this throws, only rolls back to the savepoint
    }
}

```

```
}  
}
```

## Node.js SAVEPOINTS

```
async function processOrderItems(client, orderId, items) {  
  const results = { success: [], failed: [] };  
  
  for (const item of items) {  
    await client.query('SAVEPOINT item_savepoint');  
    try {  
      await client.query(  
        'INSERT INTO order_items (order_id, product_id, qty) VALUES ($1, $2,  
$3)',  
        [orderId, item.productId, item.quantity]  
      );  
      const inv = await client.query(  
        'UPDATE inventory SET qty = qty - $1 WHERE product_id = $2 AND qty  
>= $1',  
        [item.quantity, item.productId]  
      );  
      if (inv.rowCount === 0) throw new Error('Out of stock');  
  
      await client.query('RELEASE SAVEPOINT item_savepoint');  
      results.success.push(item.productId);  
    } catch (err) {  
      await client.query('ROLLBACK TO SAVEPOINT item_savepoint');  
      results.failed.push({ productId: item.productId, reason: err.message });  
    }  
  }  
  return results;  
}
```

## Retry Logic for Serialization Failures

### SQLSTATE Codes

| SQLSTATE | Name                  | Meaning                                      |
|----------|-----------------------|----------------------------------------------|
| 40001    | serialization_failure | SSI detected a conflict                      |
| 40P01    | deadlock_detected     | Deadlock between transactions                |
| 23505    | unique_violation      | Unique constraint violated (no retry needed) |

### Python Retry Decorator

```
import time  
import psycopg2
```

```

import functools
import random

def retry_on_serialization_failure(max_retries=5, base_delay=0.1):
    """
    Decorator that retries a function on PostgreSQL serialization failure (40001)
    or deadlock (40P01) using exponential backoff with jitter.
    """
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return func(*args, **kwargs)
                except psycopg2.errors.SerializationFailure as e:
                    if attempt == max_retries - 1:
                        raise
                    delay = base_delay * (2 ** attempt) + random.uniform(0, 0.05)
                    print(f"Serialization failure, retrying in {delay:.3f}s (attempt
{attempt+1})")
                    time.sleep(delay)
                    # IMPORTANT: rollback before retry
                    conn = args[0] if args and hasattr(args[0], 'rollback') else
None

                    if conn:
                        conn.rollback()
                except psycopg2.extensions.TransactionRollbackError as e:
                    # Also catches deadlocks (40P01)
                    if attempt == max_retries - 1:
                        raise
                    delay = base_delay * (2 ** attempt) + random.uniform(0, 0.05)
                    time.sleep(delay)
                    if conn:
                        conn.rollback()

            return wrapper
        return decorator

@retry_on_serialization_failure(max_retries=5, base_delay=0.05)
def transfer_funds(conn, from_account_id, to_account_id, amount):
    with conn.cursor() as cur:
        cur.execute("""
            UPDATE accounts SET balance = balance - %s
            WHERE id = %s AND balance >= %s
            """, (amount, from_account_id, amount))
        if cur.rowcount == 0:
            raise ValueError("Insufficient funds or account not found")

        cur.execute("""
            UPDATE accounts SET balance = balance + %s WHERE id = %s
            """, (amount, to_account_id))

```

```
conn.commit()
```

## Java Spring Retry on Serialization Failure

```
// pom.xml dependency: spring-retry
@Configuration
@EnableRetry
public class RetryConfig {}

@Service
public class TransferService {

    @Transactional(isolation = Isolation.SERIALIZABLE)
    @Retryable(
        value = {CannotSerializeTransactionException.class,
DeadlockLoserDataAccessException.class},
        maxAttempts = 5,
        backoff = @Backoff(delay = 100, multiplier = 2, random = true)
    )
    public void transferFunds(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findByIdWithLock(fromId);
        Account to = accountRepository.findByIdWithLock(toId);

        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException();
        }
        from.setBalance(from.getBalance().subtract(amount));
        to.setBalance(to.getBalance().add(amount));

        accountRepository.save(from);
        accountRepository.save(to);
    }

    @Recover
    public void recoverTransfer(CannotSerializeTransactionException e, Long fromId,
Long toId, BigDecimal amount) {
        // Called after all retries exhausted
        log.error("Transfer failed after retries: from={} to={} amount={}", fromId,
toId, amount);
        throw new TransferFailedException("Transfer could not be completed due to
contention", e);
    }
}
```

## Node.js Retry Pattern

```
const SERIALIZATION_FAILURE = '40001';
const DEADLOCK_DETECTED = '40P01';
```

```

async function withRetryTransaction(pool, callback, { maxRetries = 5, baseDelayMs =
100 } = {}) {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN ISOLATION LEVEL SERIALIZABLE');
      const result = await callback(client);
      await client.query('COMMIT');
      return result;
    } catch (err) {
      await client.query('ROLLBACK');
      const isRetryable = err.code === SERIALIZATION_FAILURE || err.code ===
DEADLOCK_DETECTED;

      if (!isRetryable || attempt === maxRetries) {
        throw err;
      }

      const delay = baseDelayMs * Math.pow(2, attempt) + Math.random() * 50;
      console.warn(`Retryable error (${err.code}), attempt ${attempt + 1},
waiting ${delay.toFixed(0)}ms`);
      await new Promise(resolve => setTimeout(resolve, delay));
    } finally {
      client.release();
    }
  }
}

// Usage
const result = await withRetryTransaction(pool, async (client) => {
  const { rows } = await client.query(
    'UPDATE accounts SET balance = balance - $1 WHERE id = $2 RETURNING
balance',
    [100, fromAccountId]
  );
  if (rows[0].balance < 0) throw new Error('Insufficient funds');
  await client.query(
    'UPDATE accounts SET balance = balance + $1 WHERE id = $2',
    [100, toAccountId]
  );
  return rows[0].balance;
});

```

## Transaction Timeout Patterns

### PostgreSQL-Level Timeouts

```

-- Statement timeout: kill individual queries exceeding limit
SET statement_timeout = '30s';

```

```
-- Lock timeout: fail immediately if cannot acquire lock within limit
SET lock_timeout = '5s';

-- Idle-in-transaction timeout: kill sessions left idle inside a transaction
SET idle_in_transaction_session_timeout = '60s';

-- Transaction timeout (PostgreSQL 17+)
SET transaction_timeout = '120s';
```

## Setting Timeouts in Applications

```
# psycopg2: set per-connection
conn = psycopg2.connect(dsn, options="-c statement_timeout=30000")

# Or per-transaction
with conn.cursor() as cur:
    cur.execute("SET LOCAL statement_timeout = '30s'")
    cur.execute("SET LOCAL lock_timeout = '5s'")
    # ... queries ...
    conn.commit()

# Connection string options
conn = psycopg2.connect(
    "postgresql://user:pass@host/db?options=-c%20statement_timeout%3D30000"
)
```

```
// Spring: per-transaction timeout (seconds)
@Transactional(timeout = 30) // rolls back after 30 seconds
public void longRunningProcess() { /* ... */ }

// HikariCP: connection-level
spring.datasource.hikari.connection-timeout=30000           # 30s to get connection
spring.datasource.hikari.idle-timeout=600000                # 10min idle before close
spring.datasource.hikari.max-lifetime=1800000               # 30min max connection age
```

```
// Node.js: set timeout on client before transaction
async function timedTransaction(pool, callback, timeoutMs = 30000) {
    const client = await pool.connect();
    try {
        await client.query(`SET statement_timeout = ${timeoutMs}`);
        await client.query(`SET lock_timeout = 5000`);
        await client.query('BEGIN');
        const result = await callback(client);
        await client.query('COMMIT');
        return result;
    } catch (err) {
        await client.query('ROLLBACK');
        throw err;
    }
}
```

```

    } finally {
      // Reset timeouts before returning to pool
      await client.query('SET statement_timeout = DEFAULT');
      await client.query('SET lock_timeout = DEFAULT');
      client.release();
    }
  }
}

```

## Application-Level Timeout with Promise.race

```

function withTimeout(promise, ms, errorMessage = 'Operation timed out') {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error(errorMessage)), ms)
  );
  return Promise.race([promise, timeout]);
}

// Usage
const result = await withTimeout(
  withTransaction(pool, async (client) => {
    return await client.query('SELECT complex_computation()');
  }),
  30000, // 30 second application-level timeout
  'Database transaction timed out'
);

```

## Common Mistakes

1. **Not rolling back on error** — leaving a connection in an error state in psycopg2 causes `InFailedSqlTransaction` errors on all subsequent queries until rollback.
2. **Holding transactions open during network calls** — calling an external API, sending an email, or calling a microservice while inside a database transaction holds a DB connection and locks for the entire duration.
3. **Using `@Transactional` on private methods** — Spring AOP proxies only intercept external calls; internal calls bypass the proxy entirely.
4. **Not retrying serialization failures** — code that uses `SERIALIZABLE` isolation will see errors; without retry logic it appears "randomly broken".
5. **Retry without rollback** — attempting to retry a transaction without calling `ROLLBACK` first leaves the connection in a failed state.
6. **Overly large transactions** — wrapping entire request handlers in a single transaction increases lock contention and hold times.
7. **Ignoring `idle_in_transaction_session_timeout`** — a bug that leaves a transaction open can hold locks indefinitely, blocking other operations.



8. **Using statement timeout for transaction timeout** — `statement_timeout` resets per statement; a long transaction with many short statements can bypass it. Use `idle_in_transaction_session_timeout` for open transaction detection.

---

## Best Practices

- Keep transactions as short as possible — acquire locks late, release early
  - Set `statement_timeout` , `lock_timeout` , and `idle_in_transaction_session_timeout` at the session level
  - Always call `ROLLBACK` before retrying a serialization failure
  - Use `SERIALIZABLE` isolation only when you need true serializability — it is more expensive
  - Prefer `READ COMMITTED` with application-level optimistic locking for most web workloads
  - Use `SAVEPOINT` for partial rollback within complex multi-step operations
  - Make transactions idempotent where possible so retry logic is safe
  - Log transaction retries — frequent retries signal high contention that warrants schema or query redesign
- 

## Performance Considerations

### Transaction Overhead

| Operation                  | Approx. Cost                                   |
|----------------------------|------------------------------------------------|
| BEGIN (no IO)              | ~0.01 ms                                       |
| COMMIT (WAL flush to disk) | 1–5 ms (HDD), 0.1–0.5 ms (SSD), <0.1 ms (NVMe) |
| ROLLBACK                   | ~0.05 ms (no WAL flush needed)                 |
| SAVEPOINT creation         | ~0.01 ms                                       |
| ROLLBACK TO SAVEPOINT      | ~0.1 ms + undo work                            |

### Connection Holds Under Load

With `default_pool_size = 20` and average transaction time of 50 ms:

```
Theoretical max throughput = 20 / 0.050 = 400 TPS
```

Reducing average transaction time to 5 ms:

```
Throughput = 20 / 0.005 = 4,000 TPS
```

Reducing transaction duration is often more impactful than adding more connections.

---

## Interview Questions & Answers

**Q1: What happens in psycopg2 if you execute a query without calling commit()?**

A: psycopg2 operates in autocommit=False mode by default, which means it automatically issues `BEGIN` before the first query. If you close the connection or cursor without calling `commit()` , psycopg2 calls

`rollback()` automatically via the context manager or destructor. All changes are lost. This is a common bug when developers forget that `psycopg2` is always in a transaction.

## Q2: Why does `@Transactional` not work on private methods in Spring?

A: Spring's `@Transactional` works via AOP (Aspect-Oriented Programming) using a proxy class that wraps your bean. When external code calls a `@Transactional` method, it goes through the proxy, which sets up the transaction. When a method calls another method within the same class (self-invocation), it bypasses the proxy entirely and calls the method directly. Private methods are not visible to the proxy at all. The fix is to inject the bean into itself or extract the inner method to a separate Spring component.

## Q3: What is the difference between `ROLLBACK` and `ROLLBACK TO SAVEPOINT`?

A: `ROLLBACK` aborts the entire transaction and undoes all changes since `BEGIN`. `ROLLBACK TO SAVEPOINT` name undoes only the changes made after the `SAVEPOINT` name was created — the transaction continues and earlier changes within the transaction are preserved. The savepoint itself can be kept (for future rollbacks) or released with `RELEASE SAVEPOINT`.

## Q4: When should you use `SERIALIZABLE` isolation level?

A: Use `SERIALIZABLE` when your application logic requires that concurrent transactions produce results equivalent to some serial execution order. Examples: double-entry accounting where the sum of all accounts must remain constant, inventory allocation where you must prevent overselling, read-modify-write cycles where you need to detect concurrent modifications. The cost is serialization failures (SQLSTATE 40001) that require retry logic, plus higher lock overhead.

## Q5: How do you handle serialization failures in a Spring application?

A: Use the `spring-retry` library with `@Retryable(value = CannotSerializeTransactionException.class)`. The transaction must be retried from the beginning — Spring's `@Transactional` proxy will start a fresh transaction on each retry. Ensure the retry method is idempotent or use a unique idempotency key to prevent duplicate inserts. Configure exponential backoff with jitter to reduce retry contention.

## Q6: What is `idle_in_transaction_session_timeout` and why is it important?

A: It is a PostgreSQL parameter that automatically terminates any session that has been idle inside a transaction for longer than the specified duration. Without it, a bug (e.g., an uncaught exception that leaves a transaction open) will hold locks indefinitely, blocking all other operations on those rows or tables. Set it to a value slightly higher than your longest expected transaction (e.g., 60 seconds for an application where no transaction should take more than 30 seconds).

## Q7: What is the `readOnly = true` flag in Spring `@Transactional` and what does it actually do?

A: It is a hint, not an enforcement. Effects: (1) Spring sets the `readOnly` flag on the JDBC connection, which the PostgreSQL driver translates to `SET TRANSACTION READ ONLY` — the server will reject any DML. (2) Hibernate disables dirty checking and flushing, reducing CPU overhead. (3) The connection can potentially be routed to a read replica if you have a routing `DataSource` configured. (4) Some transaction managers (not all) optimize lock acquisition.

## Q8: Why should you not call an external HTTP API inside a database transaction?

A: The database connection is held open for the entire duration of the transaction. An external HTTP call might take 100ms–30s (or time out entirely), during which: the connection is unavailable for other work, any locks acquired in the transaction prevent other transactions from proceeding, and if the HTTP call fails, you must

decide whether to commit or roll back changes already made to the database. The database has no way to participate in the distributed transaction protocol. Always perform external calls outside of database transactions, or use the Outbox pattern.

---

## Exercises with Solutions

### Exercise 1: Fix the Node.js Transaction

**Problem:** This code has multiple issues. Identify and fix them.

```
async function transferMoney(fromId, toId, amount) {
  const client = await pool.connect();
  await client.query('BEGIN');
  await client.query('UPDATE accounts SET balance = balance - $1 WHERE id = $2',
[amount, fromId]);
  await notificationService.sendEmail(toId, 'Transfer initiated'); // external
call
  await client.query('UPDATE accounts SET balance = balance + $1 WHERE id = $2',
[amount, toId]);
  await client.query('COMMIT');
  client.release();
}
```

**Solution:**

```
async function transferMoney(fromId, toId, amount) {
  // 1. Move external call OUTSIDE transaction
  // 2. Add try/catch/finally for proper error handling
  // 3. Always release in finally block

  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    await client.query('SET LOCAL lock_timeout = \'5s\'); // prevent deadlock
hangs

    const result = await client.query(
      'UPDATE accounts SET balance = balance - $1 WHERE id = $2 AND balance >=
$1 RETURNING balance',
      [amount, fromId]
    );
    if (result.rowCount === 0) {
      throw new Error('Insufficient funds or account not found');
    }

    await client.query(
      'UPDATE accounts SET balance = balance + $1 WHERE id = $2',
      [amount, toId]
    );

    await client.query('COMMIT');
```

```

    } catch (err) {
        await client.query('ROLLBACK');
        throw err;
    } finally {
        client.release(); // ALWAYS runs, even if ROLLBACK throws
    }

    // External call AFTER successful commit
    await notificationService.sendEmail(toId, 'Transfer completed');
}

```

## Exercise 2: SAVEPOINT for Partial Failure

**Problem:** Write Python code that processes a batch of events, skipping individual failures without rolling back successful ones.

**Solution:**

```

def process_events_batch(conn, events: list[dict]) -> dict:
    results = {'processed': 0, 'failed': 0, 'errors': []}

    with conn.cursor() as cur:
        for event in events:
            cur.execute("SAVEPOINT event_sp")
            try:
                cur.execute("""
                    INSERT INTO processed_events (event_id, payload, processed_at)
                    VALUES (%s, %s, NOW())
                    """, (event['id'], psycopg2.extras.Json(event['data'])))

                cur.execute("""
                    UPDATE event_queue SET status = 'done' WHERE id = %s
                    """, (event['id'],))

                cur.execute("RELEASE SAVEPOINT event_sp")
                results['processed'] += 1

            except Exception as e:
                cur.execute("ROLLBACK TO SAVEPOINT event_sp")
                results['failed'] += 1
                results['errors'].append({'event_id': event['id'], 'error': str(e)})

    conn.commit()

    return results

```

## Cross-References

- [01\\_connection\\_pooling\\_deep.md](#) — Transaction pooling mode and SAVEPOINT support
- [02\\_orm\\_optimization.md](#) — ORM transaction management (Django `atomic`, SQLAlchemy `session`)

- `05_api_database_patterns.md` — Optimistic locking pattern (version column)
- `06_multi_tenant_architectures.md` — RLS and transaction-scoped tenant context
- `09_java_spring_postgresql.md` — Spring `@Transactional` deep dive, HikariCP timeouts
- `10_nodejs_postgresql.md` — Node pg transaction blocks and TypeORM transactions
- `11_python_postgresql.md` — psycopg3 async transactions, SQLAlchemy session transactions